FARM · π0.5

Training a Domain-Robust Manipulation Policy

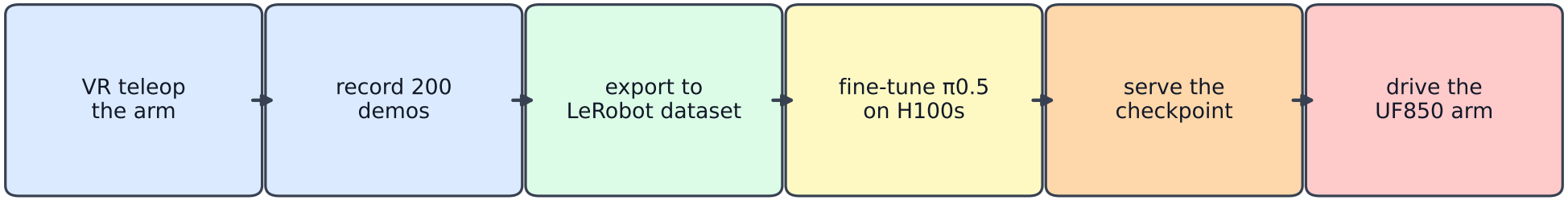
Project briefing — the goal, the diagnosis, and exactly what's running

UF850 arm · bottle pick-and-place · overnight 4× H100 run

The deployed policy fails live not because the model is weak, but because the deploy ROOM differs from the training room. You can't fix a domain shift with a better fine-tune — so we train for robustness to appearance change instead.

Prepared autonomously during the run. See farm_pi05_domain_robustness.pdf for results.

What FARM is — the loop



π0.5 = a ~3.3B-parameter vision-language-action model (a pretrained generalist robot policy).

The concrete goal tonight

Produce the best deployable policy we can, on 4 GPUs overnight, for the two bottle tasks ('put the bottle on the box' and 'move it to the desk') — one that actually works when run on the arm, not just one that scores well offline.

The data we have

200 teleop demos / ~59k frames / 2 tasks, all recorded in ONE environment (a living room). We are NOT collecting new data tonight — we only have these demos to work with.

The problem we're solving

The symptom

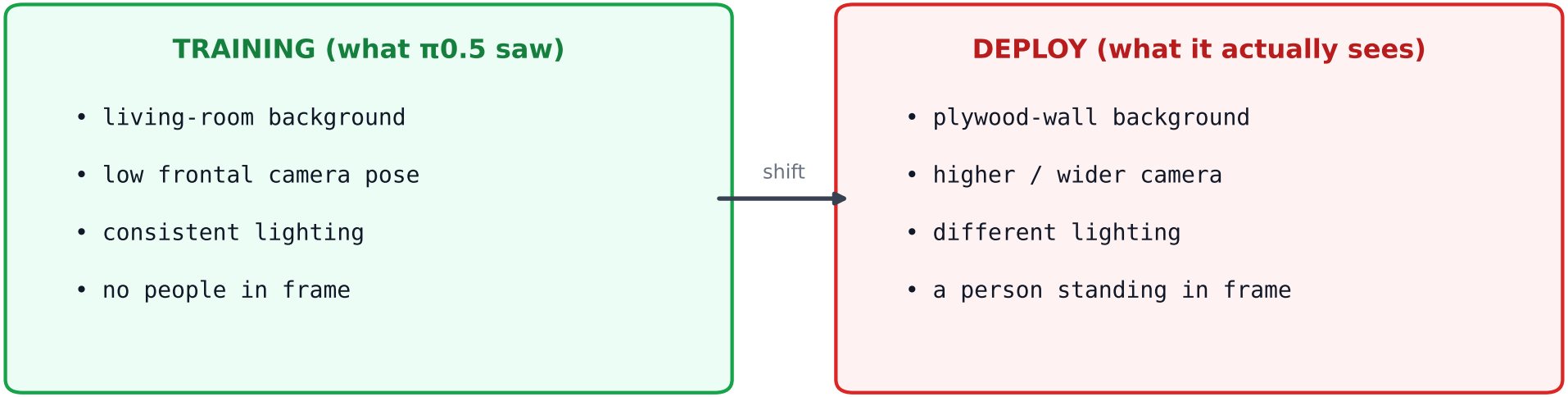
The currently-deployed full fine-tune gets ~30% success on the TRAINED bottle tasks, fails completely outside them, and moves jerkily.

The decisive clue: offline evaluation scored under 1 degree of action error and looked great — yet the live policy on the arm was effectively blind. A model that 'looks perfect' offline but fails live is almost never a capacity problem; it's an INPUT problem — at run time it is seeing something different from what it trained on.

So the question becomes

What is different between the frames the policy trained on and the frames it sees live? (Answer on the next page.) Everything we do tonight follows from that answer.

The diagnosis — a domain shift, not a weak model



The vision tower was tuned on the living-room look. In a different room it sees out-of-distribution images and emits canned / garbage motion. This is why the deployed full fine-tune AND a later GSE model both failed live in the SAME way.

Why offline evaluation lied

Offline eval replayed the ORIGINAL living-room JPEGs, so it never exercised the new camera or room — it scored sub-1° error and looked perfect while the live policy was effectively blind. It measured memorization of the training frames, not transfer to the deploy scene.

Secondary contributors (smaller, but real)

- Full fine-tuning all 3.3B params on just 2 tasks memorizes two canned trajectories AND erases π0.5's broad pretrained priors.
- Only 2 fixed prompt strings degrade the language pathway — and the two tasks are distinguishable ONLY by the prompt.

PRINCIPLE: no fine-tuning architecture fixes a domain shift. You either change the data (collect demos in the new room) or make the model robust to appearance change.

The strategy — why we're doing it this way

We cannot collect demos in the deploy room tonight (no physical access). So we use the two training-time levers that attack the diagnosed causes using ONLY the data we have:

1. Visual domain randomization (the main lever)

Aggressively perturb the training images every step — brightness, contrast, saturation, HUE, per-channel gamma, occasional grayscale, blur, and stronger crop/rotate. This forces the vision encoder to key on the TASK (where is the bottle, where is the box) rather than the living-room look. A new room then looks like just one more perturbation it has already learned to ignore. This is the standard sim-to-real / cross-environment mitigation.

2. Base preservation via GSE

Fine-tune in a way that KEEPS π0.5's pretrained vision/language priors (which are inherently robust to scene changes), instead of overwriting them like a full fine-tune.

3. Prompt paraphrasing

Sample varied phrasings of each task ('put the bottle on the box', 'place it onto the box', ...) so the language pathway stays healthy and the two tasks — which are ONLY distinguishable by prompt — don't blur together.

Exactly what we're training — a controlled 2×2

All four cells are GSE (base-preserving), same recipe (batch 128, 3000 steps ≈ 6.5 epochs, 4×H100, ~70 min each). Only the two augmentation knobs change — so any difference is attributable to the knob, not luck. The 'neither' cell is the already-trained vanilla GSE.

	prompt-aug OFF	prompt-aug ON
default visual aug	**gse (vanilla)** already trained	**gse_prompt** isolates language
HEAVY domain randomization	**gse_aug** isolates vision	**gse_robust ★** FLAGSHIP — both

Why GSE (not a plain full fine-tune)

GSE = Generalized & Specialized Experts. It SVD-splits each weight into a PRESERVED part (keeps π0.5's robust pretrained vision/language priors) and an ADAPTED part (learns our task). A full fine-tune instead overwrites those priors — which is part of why it failed live.

We also keep the deployed full fine-tune (20k steps) in the comparison as the reference baseline.

How we judge it — clean fit vs. domain-shift robustness

Both scores are open-loop (no robot): each model replays the 6 held-out episodes and its predicted action is compared to the recorded ground truth.

Clean fit (the misleading metric)

Error on the ORIGINAL frames. This is what looked perfect before while the live policy failed — it mostly measures memorization, not transfer. We report it, but it is not the metric that matters.

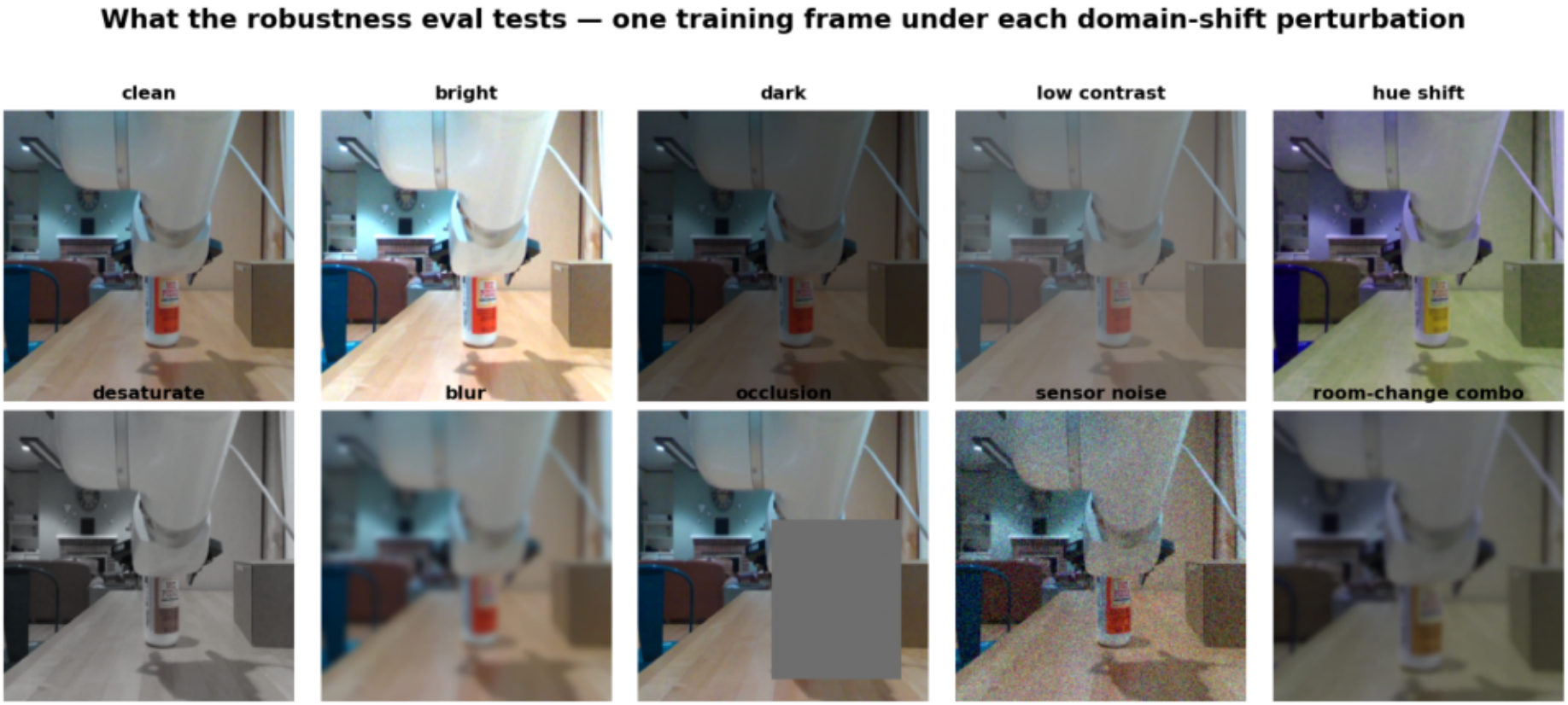
Domain-shift robustness (the headline)

We PERTURB each frame (see the gallery, next page) — dark, bright, hue-shift, blur, occlusion, a stacked 'room-change combo' — BEFORE asking the policy to act, and measure how much the action degrades. This is a proxy for the real room change. Perturbations are seeded so every model sees byte-identical inputs (a fair comparison).

⚠ The honest caveat

Synthetic perturbations are a PROXY, not the real plywood room. A model that holds up here is more LIKELY to transfer, but the only true validation is live deployment / collecting a few demos in the target room. We are explicit about this so the numbers aren't oversold.

What the robustness eval actually tests



One real training frame (base camera) shown under each perturbation condition. The eval asks: when the scene looks like this instead of the clean original, how much does each model's predicted action drift? The 'room-change combo' (bottom-right) stacks several to mimic a genuine room change.

What is running right now

One SLURM job on 4× H100

- Builds the container once, computes normalization stats, then trains the three GSE variants back-to-back (3000 steps each, ~70 min each), streaming checkpoints to a HuggingFace repo per variant.
- Flagship (gse_robust) is already done: loss fell 0.0744 -> 0.0019, matching vanilla GSE's fit despite the heavy augmentation (the augmentation was 'free' — no loss of task fit).

The engineering behind it

- patch_openpi_aug.py — makes openpi's image augmentation env-controlled and adds the heavy domain-randomization recipe (backward-compatible; the serving path is unchanged).
- farm_prompt_aug.py — env-gated prompt paraphrasing (identity at serve time).
- eval_robust.py — the domain-shift robustness eval. make_report.py — the results PDF.
- We caught and fixed a NaN-loss bug (gamma applied to negative pixels) in a 20-step smoke test BEFORE committing any of the 4-GPU budget.

Budget & sequence

- 4 GPUs at a time, ~20 of the 40 GPU-hours. Sequence: train (~4.5h) -> one 1-GPU eval pass over all 5 models (clean + robust) -> final report PDF in this folder.

What to expect, and the honest limits

The hypothesis we're testing

On clean frames every model looks fine. Under the perturbations, the flagship (heavy aug + prompt aug, base-preserving) should degrade the LEAST, while the deployed full fine-tune collapses — which would both explain the live failure and point to the fix.

If confirmed

Deploy the flagship (NoahWeiss/farm_uf850_pi05_gse_robust, step-2999). The serve command is in the results PDF and DEPLOYMENT.md.

What this does NOT claim

Domain randomization NARROWS the train/deploy gap; it does not close it. It cannot substitute for in-domain data. The real, durable fix remains: collect a few demos in the plywood room (ideally across a couple of lighting conditions) and fine-tune on them.

The one habit that would have caught this sooner

Before blaming the model, compare a LIVE camera frame against a training frame. The whole domain-shift diagnosis came from literally looking at the two images side by side.